

This is a fantastic question. The interaction between these parameters is the hardest part of Deep RL to grasp, but it is the secret to making the model actually learn instead of just memorizing.

Let's break this down using a simple analogy: **Studying for an exam.**

1. The Data Collection Phase (Gathering Notes)

- **NUM_ENVS = 64**: You clone your AI agent into 64 parallel universes.
- **NUM_STEPS = 256**: You drop each clone at a random starting date in your 17-year history. They each trade forward for 256 days and record what happened.
- **The Buffer**: The clones come back and combine their notes.
 - $64 \text{ clones} \times 256 \text{ days} = \mathbf{16,384}$ days of experience.

How does this relate to your 17 years (4,524 days) of data? Because the clones collected 16,384 days of data, and your total history is only 4,524 days long, **your agent just experienced the entire 17-year history about 3.6 times in a single sweep!**

2. The Learning Phase (Updating the Brain)

Now the agent sits down to learn from the 16,384 days of notes.

- **MINI_BATCH_SIZE = 4096**: The neural network can't process all 16,384 days at once (it would crash your GPU memory). So, it shuffles the notes and splits them into stacks of 4,096.
 - $16,384 \div 4,096 = \mathbf{4}$ stacks (or batches).
 - The neural network looks at a stack, calculates its mistakes, and updates its brain weights **1 time per stack**. So, reading all the notes takes **4 weight updates**.
- **update_epochs = 6**: (Note: In your `02_RLVR...` code Cell 11, this is currently hardcoded to 4, but let's use 6 as you asked).
 - This is how many times the agent re-reads the *entire* collection of notes before throwing them away.
 - If it reads the 4 stacks, updates its brain 4 times, and does this 6 times (`update_epochs=6`), **the brain weights are updated 24 times** during this learning phase.

3. The Outer Loop (Going back to the market)

- **NUM_EPOCHS = 75**: This is the outer grid parameter. Once the brain is updated, the agent throws away the old notes, goes back to the market, and collects a brand new batch of 16,384 days. It repeats this whole cycle 75 times.

Answering your specific questions:

How many times does the agent see the 17 years of data?

- In one loop, it sees the whole 17 years **~3.6 times**.
- Over 75 **NUM_EPOCHS**, it sees the whole 17 years about **270 times**.

How many times do the weights get updated total?

- 4 updates per pass $\times$ 6 passes (update_epochs) $\times$ 75 outer loops = **1,800** total weight updates.

Can we update the weights too much? YES. This is the #1 cause of failure in PPO. If `update_epochs` is too high (e.g., 10+), the agent reads the exact same notes over and over. Instead of learning general rules like *"buy stocks with good momentum,"* it just memorizes the data: *"Oh, it's March 2009, I should buy Apple."* When you test it on new data (2022+), it will fail miserably because it memorized the past instead of learning how to trade. This is called **Overfitting**.

My Beginner-Friendly Recommendation for your Grid Sweep:

Because your dataset is 4,524 days, gathering 16,384 days at a time is too aggressive. You want the agent to collect a *smaller* batch of notes, learn a little bit, and go back to the market.

Change your grid to this:

- `NUM_ENVS = 16`
- `NUM_STEPS = 256`
- *(This creates a buffer of 4,096 days. That is ~90% of your 17-year history per loop, which is perfect!)*
- `MINI_BATCH_SIZE = 1024` *(4 stacks per loop)*
- Leave `update_epochs = 4` *(In Cell 11 of your code. 4 is the industry standard for PPO to prevent overfitting).*
- `NUM_EPOCHS = 75` *(It will now smoothly traverse and learn the history without memorizing it).*